

The SKG-IF Model and its Extension Workflow in Practice

Final review draft — v0.10 (2026-06-29)

Title	The SKG-IF Model and its Extension Workflow in Practice
Author	Daan Broeder
Status	draft — v0.10
Date	2026-06-29
Audience	OSTrails consortium partners implementing SKG-IF / SKG-IF maintainers / RDA SKG-IF WG
Purpose	Hands-on technology report on the costs and benefits of the SKG-IF extension-creation process, grounded in the ext-srv (Service) extension experience, investigating potential workflow alternatives
DOI	https://doi.org/10.5281/zenodo.20692571

About this document

This is a hands-on technology report on the SKG-IF extension-creation process, grounded in the author's direct experience with the ext-srv (Service) extension developed within OSTrails and informed by multi-community discussion documents from the OSTrails pilot communities. It is written as a contribution intended for OSTrails internal circulation, with the option to be reworked into an RDA SKG-IF WG input and/or public paper.

This report is intended for two audiences: research community experts considering the use of SKG-IF, and participants in the RDA SKG-IF Working Group, audiences that overlap.

For readers who cannot engage with the full report, Sections 0, 1, 2 and 9 form a self-contained unit: background, findings, and recommendations without the detailed case-study material of Sections 3–8.

1. Executive summary

The SKG-IF Interoperability Framework provides a core model and a formally specified extension workflow. Applied in practice for the ext-srv Service extension, and cross-read against the commentary from a number of the OSTrails pilot communities (CESSDA, CLARIN, SSHOMP, ESCAPE, ESRF-PaNOSC), the extension exercise leads to a number of observations and recommendations presented in this report.

It should be said that from the start of the SKG-IF extension work the author and several collaborators had doubts about the complexity of the SKG-IF extension mechanism, specifically whether the ontology-grounding efforts are compensated by real benefits. The issues below are what we consider relevant in that respect.

- The bibliographic origins of the core model, created in the SKG-IF RDA WG by a number of Scholarly Knowledge Graph initiatives, leave the core model over-specified in places where research communities can use lighter structures, while it is lacking for the research life-cycle

(no first-class Services, Instruments, Facilities, Projects, Portals, Communities, Research Infrastructures).

- The OSTrails pilot communities that did contribute commentary on exchange mechanisms, SSHOC/SSHOMP and PaNOSC/ESRF most consistently, converge on two preferences: API over ZIP for bulk exchange, and JSON-LD over plain JSON for serialisation. The JSON-LD preference is a convergent signal from the commentary documents rather than a surveyed consensus; individual community conversations may differ.
- The OWL ontology layer's operational role is narrower than the workflow assumes. No OSTrails implementer consumes the OWL at reasoning level. The ontology earns its keep through SHACL generation and formal semantic grounding of SKG-IF terms against upstream vocabularies, both for human readers. The JSON-LD context, which is what the artefact data producers actually work with, is hand-maintained separately from the OWL.
- The current workflow is labour-intensive. Keeping several artefacts in sync without automatic tooling is problematic; expert support and review depend on two WG chairs, only one of whom is funded through OSTrails; and the hope for community review is feasible only for domain-specific questions, not for ontology design.
- Lighter extension procedures are conceivable. Using LinkML (Section 7.3) or dropping the mandated OWL layer entirely (Section 7.1), is possible without losing interoperability.
- Structural critiques of the core model have been raised in OSTrails-adjacent discussion [internal reference on file, not for circulation]: arguments that the model is not yet a "true graph model" — Venues, Data Sources and Grants inflate the node set with objects that are arguably better modelled as typed relationships. Section 8 develops the technical reasoning.

The case that can be made for the current workflow is that the OWL layer formally grounds SKG-IF terms in established ontologies, making their meaning verifiable beyond the framework's own definitions and perhaps yet unplanned future use of reasoning and inference. The WG has not explicitly argued against lighter alternatives beyond that. The current stack reflects how the framework was designed.

This report's position is that the case for the OWL layer is understandable but not yet materialising: no OSTrails implementer uses the OWL at reasoning level, and only two implementations are in active production. The recommendations in this document aim to make the workflow proportionate to that reality, not to dismantle it.

A lighter extension tier that does not mandate a new OWL ontology, and a core model better fitting for research communities beyond the bibliographic scope, are the central recommendations. The full argument is in Sections 5–8, the recommendations are in Section 9.

2. Background

2.1 Origins, bibliographic focus, and research community mismatch

SKG-IF (Scholarly Knowledge Graph Interoperability Framework) was initiated by the RDA Interest Group on Open Science Graphs for FAIR Data and handed over in June 2023 to the RDA Working Group on SKG-IF.¹

The WG operates with two co-chairs (Andrea Mannocci, Silvio Peroni), a rolling monthly / bi-weekly meeting cadence and a rolling-agenda-and-minutes document as an authoritative record.

Development and discussion take place primarily on the SKG-IF GitHub organisation (github.com/skg-if).

¹ The RDA WG uses "Scholarly"; "Scientific" is the broader framing preferred in OSTrails and similar research infrastructure contexts where the research life-cycle is not primarily bibliographic

The SKG-IF core model defines six entity types — Research Product, Agent, Venue, Topic, Grant, Data Source — and expresses them through a layered technical stack (OWL ontology, SHACL, JSON-LD context, OpenAPI specification). It was designed primarily by parties whose principal focus is bibliographic — OpenAIRE, DataCite, OpenCitations and similar — and the entity set reflects those origins. Testing whether the SKG-IF can be adapted to and used for the research domain beyond the bibliographic core is itself a specific objective of the OSTRails project and was one of the reasons why several of the research communities joined the project.

The core model fits OSTRails' research communities imperfectly, and the mismatch runs in two directions:

1. The model is over-specified or biased bibliographically. Several modelling choices that make sense in a publication-centric graph do not carry over into research-community practice. A multi-community analysis spanning CESSDA, CLARIN, SSHOC, ESCAPE and PaNOSC of SKG-IF [§10.1] documented this in detail; summarised here in paraphrase:
 - identifier requirements force generation and restructuring of community entity identification to fit the `local_identifier` pattern;
 - the contribution vocabulary for research products (CRediT) lacks roles for research infrastructure operation, e.g. provider, helpdesk, local facility contact;
 - manifestation versioning does not model the embargo-expiry versioning pattern common in social science archives, where a restricted version is succeeded by a full release when the embargo period ends;
 - grant, topic, and organisation entity metadata requirements are too heavyweight for communities that record only a funder name, a keyword string, or an organisation name;
 - organisations could originally not be modelled as contributors. As this is an essential need, it was quickly resolved.
2. At the same time the model is under-specified for the research life-cycle. The bibliographic core does not model a number of entity types that are first-class in the day-to-day operation of research communities. A community inventory compiled in the same multi-community analysis [§10.1] identified: Portal, Community, (Research) Infrastructure, Services and tools (including the service-vs-software distinction), Instruments, Facilities, Projects, detailed dataset descriptions (observation type, coverage, measurable units), and ground-truth / calibration datasets. This was discussed in shared documents to which the SKG-IF RDA WG chairs were invited [Comments on the SKG-IF model, §10.1].

The ESCAPE community analysis adds data-access services and APIs (dataset / distribution / catalogue / service in DCAT terms), Instruments and Facilities with PIDs, and an observation-technique typology. Additionally, communities such as CESSDA identified missing core fields, unit of analysis, sampling procedure, geographic coverage, which do not require new entity types but do require existing core entity modification. Together these constitute the entity and field set that a research-life-cycle knowledge graph needs to cover, but which the current core does not.

Importantly, the framework already provided an extension mechanism which was used only once for a minimal extension. It is an open question whether that mechanism was designed with the type of new-entity extension we require in mind; it seems rather aimed at adaptation and reuse of existing entities. Answering that question is precisely one of the things ext-srv is testing in practice, adding a new entity type through the prescribed mechanism end-to-end, providing a basis for further discussion or alternative strategies, alongside the accurate modelling of the Service concept itself.

2.2 SKG-IF and metadata interoperability

Interest from research communities in SKG-IF is largely framed around metadata interoperability, an issue communities continue to wrestle with. The CLARIN metadata infrastructure (CMDI),² for example, was based on a similar component-metadata approach to achieving interoperability within the CLARIN domain.

The prospect that SKG-IF could play an analogous cross-community role is part of what drew communities into OSTRails, and it is also the context in which the present author and others noticed the tension between the framework’s bibliographic origins and the metadata realities of research communities.

2.3 OSTRails context and the ext-srv case study

OSTRails is the context in which this work is situated. The author is co-proponent of the Service (srv) extension together with Menzo Windhouwer, Matej Ďurčo, Renaud Duyme and Baptiste Cecconi.

The ext-srv extension was deliberately scoped wider and at a higher level of semantic detail, in the Service entity type, than the strict minimum needed by any single community: the intent was to exercise the entire SKG-IF extension machinery (OWL + SHACL + JSON-LD + OpenAPI overlay + examples + governance) so that its true costs and benefits could be observed rather than asserted.

The OSTRails project itself already officially anticipates extension activity of this kind. Use Case III of the OSTRails Pilots-implementation-actions document [§10.1] names the creation of extensions for “new services, instruments, cultural heritage objects, etc.” as a foreseen path for pilots; the extension workflow is thus part of the project’s own plan, not an ad-hoc addition.

2.4 The report motivations

This report grew out of three questions that could only be answered by actually working and extending the SKG-IF: Is a new-entity SKG-IF extension feasible without deep semantic-web expertise? Is the effort proportionate to the result? And is SKG-IF flexible enough to model research-community entities beyond the bibliographic core? The ext-srv implementation work, and the multi-community commentary, put those questions to the test.

The labour-intensity concerns three distinct things: (1) the technical effort of authoring and keeping in sync the multiple artefacts that the workflow requires (§5.3); (2) the governance bottleneck created by concentrating the review of those artefacts with two experts whose time is partly voluntary (§5.4); and (3) the uncertainty, for a prospective extension author, about which tier of the workflow is actually appropriate for their use case, because that triage is currently informal and chair-side (§5.4.1, §7.6).

In our work and discussions some potential benefits of an OWL ontology specification have been identified: for example, RDF triple store configuration (relevant for one of the OSTRails Pilots), and enabling future query expansion in federated search scenarios. These benefits are real, but conditional on downstream tooling that does not yet exist for the OSTRails community at scale.

3. The SKG-IF core model (brief)

This section gives a brief reference overview of the SKG-IF core model; the extension workflow (Section 4) and the assessment (Sections 5–8) presuppose this structure.

Summarising the main SKG-IF model:

² Broeder, D., Kemps-Snijders, M., Van Uytvanck, D., Windhouwer, M., Withers, P., Wittenburg, P. & Zinn, C. (2010). A data category registry- and component-based metadata infrastructure. LREC 2010.

- Six core entity types: Research Product, Agent, Venue, Topic, Grant, Data Source.
- Technical stack: OWL ontology (FaBiO-based, SKG-O) + SHACL shapes + JSON-LD context + OpenAPI 3.1 specification. Every core extension mirrors this four-layer structure.
- Context stack: the JSON-LD context is layered — core context (v1.1.0) → API context (v1.0.0) → @base (provider-specific URL space) → extension context. The order is load-bearing; layers must not be collapsed or reordered.
- Identifiers: entities are referenced by `local_identifier` — a string scoped to a provider-specific URL space, minted freely by the implementer. The `w3id.org/skg-if/` path is recommended; community-own PID provisioning is permitted where operationally preferable.
- Versioning and publishing: all artefacts (ontology, SHACL, context, API) are version-stamped and published under `w3id.org/skg-if/` with a `current/` alias that always resolves to the latest stable version. Extensions follow the same convention under their own `w3id.org` subpath.
- API design principle: identifiers by default, entities inlined only when requested (`?expand=true`).
- Extensions: add new entity types and/or properties to the core, expressed through the same artefact stack. The extension mechanism is the subject of Section 4; its costs and benefits are assessed in Sections 5–8.

The architecture is coherent in intent, each layer serves a stated purpose, but distributed in execution: the same information is encoded across five artefacts that must be kept manually in sync, and some generation steps rely on non-standard conventions (notably SHACL generation from free-text annotations embedded in the OWL file). Whether each layer individually justifies that cost is what Section 6 examines.

4. The official extension-creation workflow

The following requirements are drawn from the extension repository template (`skg-if/interoperability-framework`) and describe what every extension author must deliver.

4.1 Mandatory repository structure

- **data-model/ontology** — One OWL ontology per version, plus a `current/` alias; serialised in Turtle (`.ttl`) and others; named after the extension acronym (e.g. `srv.ttl`).
- **data-model/shacl** — SHACL shapes per version, plus `current/` alias, file named `shacl.ttl`.
- **context** — JSON-LD context per version (`skg-if.json`) plus `current/` alias.
- **interoperability-framework** — Prose description of how the extension extends core entities and introduces new ones.
- **api** — Documentation of API customisation; typically an OpenAPI overlay.
- **examples** — JSON-LD example records compliant with the extension.

Note on mandated ontology deliverables: the extension template lists two required outcomes (`ext-tmpl data-model/index.md`, "Expected outcomes"): (1) OWL/RDF ontology files in four serialisation formats (`.ttl`, `.nt`, `.xml`, `.json`), needed for content negotiation behind `w3id.org`; and (2) an `.html` documentation page of the ontology. Widoco is the tool suggested in the template for producing both in a single run, and is substantially easier than the alternatives (Protégé export + manual editing). Graffoo diagrams are not part of the mandatory template structure: the extension template contains no mention of Graffoo, but the SKG-IF core documentation uses Graffoo diagrams throughout, and `ext-ra-skg` follows the same convention. The Graffoo diagram is assessed in Section 6.8.

4.2 Versioning and permanent URLs

Extensions use semantic versioning; the extension repo template (ext-tmpl) auto-generates w3id.org permanent URLs (e.g. `.../ontology/`, `.../context/skg-if.json`, `.../validation/shacl/`). The extension template repository is configured as a Jekyll site hosted on GitHub Pages (skg-if.github.io/<acronym>/). The prescribed directory structure of §4.1 is the Jekyll site layout; placing the semantic artefacts (`.ttl`, `.nt`, `.xml`, `.json`, `.html`) in the specified paths makes them automatically available at the corresponding GitHub Pages URLs, and the w3id.org permanent URLs redirect to those pages. Content negotiation for the four ontology serialisation formats is wired through this same mechanism. This means the repository structure is load-bearing in two senses: it organises files for developers, and it defines the public URL space for semantic consumers.

4.3 Validation pipeline (implementer-authored, not mandated)

The SKG-IF core does not at present provide a tool for cross-artefact consistency checking of extensions. The 8-step validation pipeline described below (dev-check.sh)³ is not part of the RDA SKG-IF workflow; it was written by the ext-srv implementers to keep the six-artefact synchronisation manageable in-repo (ontology, SHACL, context, overlay, prose spec, examples — see §5.2). It is reported here as an illustration of the cross-artefact checking the official workflow does not provide, and because it has become a de facto part of ext-srv maintenance (tools: §10.6).

- Ontology syntax check (riot).
- SHACL generation from srv.ttl via shacl-extractor.
- Cross-file alignment: ontology ↔ context ↔ SHACL ↔ OpenAPI overlay + label check.
- Generate consolidated OpenAPI spec (core + overlay).
- Spectral lint on core spec and consolidated spec.
- Context compatibility check (all srv_* properties declared).
- Start local Docker stack.
- API health checks (20 validation tests against Prism mock server).

4.4 Approval and governance path

Proposals are submitted informally to the RDA SKG-IF WG chairs, who assess scope and fit with the existing model. On acceptance the extension is initialised from the extension repo template (ext-tmpl) under the skg-if/ GitHub organisation, versioned from 1.0.0 onwards. Advertisement is via inclusion in the SKG-IF extensions index; deprecation was not discussed yet.

A note on sources for Section 4.4 and for the timeline in Section 5.4: the GitHub issues tracker under skg-if/extensions under-reports the actual deliberation of proposals, because proposals are discussed and prioritised at WG meetings rather than in the tracker. The rolling-agenda-and-minutes document of the RDA SKG-IF WG is therefore a more reliable primary source than the issue tracker for what the WG actually decided about a given proposal, and it is what this report uses.

The RDA WG output (Mannocci, Peroni & SKG-IF WG, 2025) adds two formal selection criteria to the proposal process: a proposal must address a collective community need rather than an individual project requirement and must not interfere with entities or APIs already defined in the SKG-IF or in other endorsed extensions. More significantly, the document states that the lifecycle of an extension falls outside the WG's core scope; no formal process covers deprecation, versioning after initial release, or promotion to core. This makes dependencies between different extensions problematic even if they are non-interfering, because an extension that builds on types or properties defined in

³ <https://github.com/skg-if/ext-srv/blob/main/dev-check.sh>

another has no recourse if that first extension is abandoned. Addressing this would require some form of central maintenance commitment for endorsed extensions.

5. Case study — the Service (ext-srv) extension

5.1 Motivation and scope

Many research infrastructures and community catalogues — ESCAPE, CLARIN, CESSDA, SSHOC — treat software services (portals, processing APIs, repositories of operational functionality) as first-class objects. The bibliographic core of SKG-IF does not model these directly. Therefore ext-srv introduces a `srv:Service` entity plus organisation/venue types and a set of properties (audience, invocation type, life-cycle status, hosting organisation, deployment-of software, keywords, ...). The property set was grounded in six independent vocabularies: CLARIN VLO CMDI, SSHOMP catalogue, ELG catalogue, CLARIN LR Switchboard, schema.org/SoftwareApplication, and the EOSC Service Profile. Cross-vocabulary evidence rather than one community's wish list.

The scoping decision was also a deliberate test of the extension mechanism itself. A strictly minimal extension (the “do the least to cover our own use case” option) would have produced a narrower property set, typically by pressing existing core classes and concepts into service even where they fitted only approximately. The ext-srv authors chose the opposite: to test the interoperability framework by extending it with richer, purpose-specific classes. The result is not claimed to be a perfect ontology; the point was to exercise the extension mechanism and observe where it resists. The ext-srv experience is therefore designed to be read as a stress test of the workflow, not as an over-eager optimist's maximum.

5.2 Components produced

- `srv.ttl` — OWL ontology (source of truth).
- `shacl.ttl` — SHACL shapes (auto-generated from `srv.ttl` by `shacl-extractor`).
- `skg-if.json` — JSON-LD context for `srv`.
- `service-overlay.yaml` — OpenAPI overlay adding `/services` endpoints.
- `Service.md` — 19 KB prose specification.
- `extended-interoperability-framework/core-extensions/` — per-entity Markdown files that specify how ext-srv extends existing core entities (Organisation, Venue) in addition to introducing Service; these prose files, not the `examples/` directory, carry the ext-srv adaptations of core entities.
- `examples/` — 120+ JSON-LD example Service records.
- `srv.graphml` — Graffoo diagram (yEd).
- Widoco-generated HTML documentation.

What each component is actually used for matters to §6's assessment. For ext-srv the operative answers are: the OWL file is the nominal source of truth for the ontology but is not consumed at reasoning time by any OSTrails implementer surveyed; the SHACL file is used by the `dev-check.sh` pipeline to validate the 120+ example records but has not yet been used at implementer sites for live instance-data validation; the JSON-LD context is consumed at serialisation time by every implementer that has produced SKG-IF output; and the OpenAPI overlay is exercised against a local mock server (see §4.3 for the pipeline and §6.4 for the value assessment). In other words, the ontology earns its keep as a documentation and generation input, not as a runtime object.

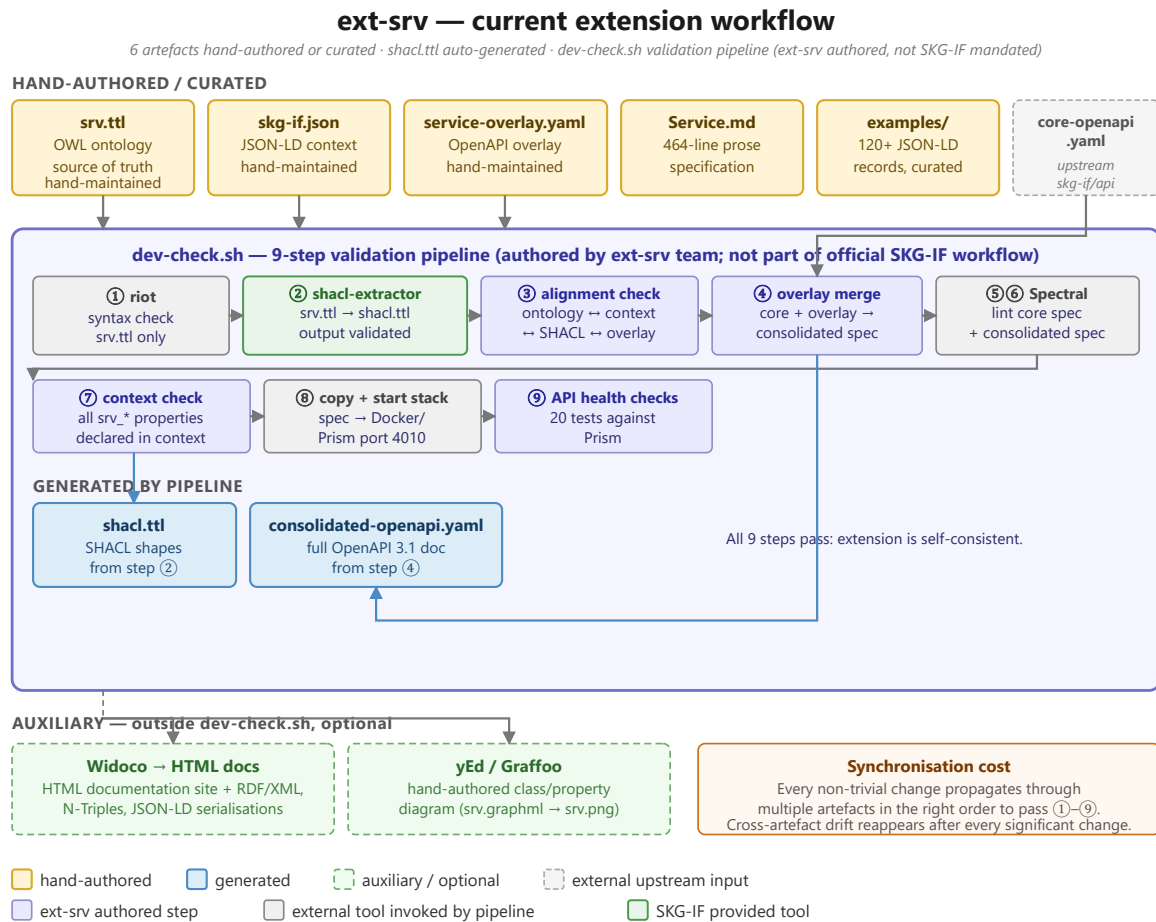


Figure 1 Current ext-srv artefact stack — six hand-authored files

5.3 Effort and synchronisation cost

The ext-srv git history records 135 commits over 8 active months (October 2025 – May 2026), across 239 files. Per-component commit counts: ontology 47, API overlay 28, prose specification 25, JSON-LD context 26, examples 23, dev-check.sh 11. No component was touched fewer than 11 times.⁴ These counts do not capture authoring time directly, but they show the tight relationship between the different artefacts and the multi-artefact synchronisation burden.

Beyond raw effort, the ext-srv experience exposes an important synchronisation cost not apparent from the extension repo template (ext-tmpl). Source-of-truth responsibility is split across several artefacts:

The OWL ontology in turtle format is the source of truth for the semantic model; the SHACL shapes are generated from it but must be regenerated after every ontology change. The JSON-LD context, OpenAPI overlay and prose spec and examples are hand-maintained separately, and every non-trivial change must propagate through each in the right order.

⁴ Fortunately the 1500 example files could be automatically generated when needed by the conversion scripts.

The `dev-check.sh` script was created precisely to manage this, but cross-artefact misalignment reappears on every significant change and the effort remains substantially higher than a single-artefact authoring process would produce.

The SHACL generation step has some opacity. The `shacl-extractor` tool (SKG-IF provided) parses cardinality constraints encoded as a mini-format inside `rdfs:comment` annotations on OWL properties and re-materialises them as formal SHACL shapes. Encoding machine-readable cardinality constraints inside `rdfs:comment` is an unconventional choice for this. The format is documented in the extractor's README, but it is a bespoke convention that an extension author must discover. The extractor did not work out-of-the-box for extensions, requiring collaboration with its developer. An important question is whether OWL decorated with informal constraints is the right path to a SHACL artefact; §7.3 and §8.1 return to this.

OWL adoption among data producers remains limited. In SKG-IF, the OWL ontology is intended as the source from which SHACL shapes and HTML documentation are generated. The JSON-LD context, the artefact data producers work with most directly, is hand-maintained separately, not derived from the OWL. No OSTRails use case in the current run requires OWL reasoning. The two closest candidates are topic-vocabulary alignment across federated endpoints (a SKOS alignment problem, not an OWL reasoning problem) and triple-store management by implementers (one pilot, with expressed doubts about this).

The OWL therefore earns its current cost through formal semantic grounding, relating SKG-IF terms to upstream ontologies (SPAR, FaBiO, FRAPO), and through SHACL generation. That grounding is for human consumption: the ontology is not proposed as a basis for machine reasoning in tools such as Protégé or GraphDB, and no merging with upstream ontologies is implied. LinkML (§7.3) offers an alternative that derives SHACL, JSON-LD context, OWL, and documentation from a single YAML schema without requiring OWL authoring directly.

5.4 Review and response timing

Beyond the author's own effort, the ext-srv experience reveals a structural feature of the extension-creation workflow: the expertise required to review extension work, particularly the OWL ontology and SHACL shapes, sits almost exclusively with the two WG chairs. The ext-srv timeline illustrates the effect:

- March 2025 — ext-srv first mentioned in the OSTRails PSC context.
- Mid 2025 — ext-srv discussed in WG; tentative acceptance of the proposal.
- 27 October 2025 — extension repository initialised from template.
- 11 November 2025 — first substantive commits (ontology, context, overlay).
- Early April 2026 — a first version offered to the WG chairs for review.
- 21 April 2026 — feedback on formal proposal format and layout.
- 29–30 April 2026 — content-relevant review session with both chairs; substantive ontology and API feedback received (see `skg-if/ext-srv/issues`). This is the first substantive review engagement after months of implementation work.
- January–May 2026 — review comments from Katja Moilanen and Menzo Windhouwer

In total four reviewers engaged with different artefacts, the two chairs and two community reviewers (Katja Moilanen and Menzo Windhouwer). The issues raised fall into four groups:

- Style and format compliance — IF page layout, use of OWL IRIs in user-facing documentation, and documentation structure. These are important for consistency with the different templates and potential adoption of the extension by groups outside the current proposers.

- Modelling issues — new organisation type classes missing their place in the class hierarchy, and a Graffoo diagram compliance error (`skos:Concept` incorrectly typed as a data type).
- JSON-LD design — referencing external SKOS vocabularies is not straightforward; SHACL cannot validate remote vocabulary membership, and the JSON-LD context has no clean mechanism for constraining values to a specific ConceptScheme.
- Framework design — the requirement to prefix all extension properties with `srv_` prevents potential namespace collisions with other extensions, but it was raised whether general properties not specific to the extension should also carry the prefix. This is a usability issue without a settled answer.

From OSTRails PSC mention to first substantive review: ~13 months; from template initialization to substantive review: 6 months. The April 29–30 session marks a positive turn: content-relevant engagement arrived and the review process is now active. These numbers set the baseline for the §9 recommendation on agreed review commitments. The concern is structural, not directed at individuals. Of the two chairs, only Andrea Mannocci (OpenAIRE) is affiliated with OSTRails; review time from the other chair is contributed outside the project. Prose, examples, and API overlay can productively be reviewed by the wider community; ontology design choices cannot be delegated to non-specialists without losing the rigour the OWL layer is meant to provide.

One caveat on responsiveness: the timeline bottleneck coexists with helpful engagement on targeted questions when both chairs were present and an immediate answer was usually provided. Resolution was sometimes further delayed when one of the chairs was absent, and the question required consultation. The bottleneck is primarily on sustained end-to-end review throughput, but meeting-attendance patterns also affect the pace of individual query resolution.

The slow progress can be seen as natural and acceptable in the context of the RDA process; however, project dependencies on its outcome (as in the case of OSTRails) make this a potential risk. In agreeing to work only with certified extensions and necessary core adaptations, a project should provide adequate resources for the RDA machinery to manage the work. Fortunately, some provisions were made for this in OSTRails, including repurposing of resources originally dedicated to Thematic Pilot 4 towards SKG-IF modelling work.

A further structural issue arises when extension work surfaces gaps in the core. There is no defined process for addressing such gaps: the chairs are understandably reluctant to act unilaterally on core changes, and the WG scheduling cycle makes even straightforward additions slow to resolve. Extension authors are left choosing between a workaround and an open-ended wait. Two cases from `ext-srv` illustrate this: first, proposing a multilingual descriptions property, useful across all entity types, not just services, was blocked from the core, forcing the prefixed workaround `srv_descriptions`. Second, proposing `is_cited_by` as a complement to the core's `cites` relation shows a further dynamic: `ext-srv` introduced `srv_is_cited_by` as a Service-level workaround, but the general case, a product-to-product inverse relation also needed by other communities, remains a stalled core change. A single core addition would have resolved both. Both cases also show how the absence of a shorter-term core-change path keeps SKG-IF anchored to its bibliographic origins rather than opening to the broader research landscape.

5.5 Other proposal trajectories — dropout and re-route patterns

The `ext-srv` timeline is one trajectory only. The rolling minutes show three patterns that calibrate what "review capacity" means in practice.

Pattern A — proponent dropout. A significant share of proposals are raised, noted by the WG, and then stall when the proponent does not return to take the next authoring step. At least six

documented cases ⁵ follow this pattern. One can read this charitably — friction filtering out under-resourced proposals — but the filter operates silently and late: proposers discover the effort after months of work, not before starting. The remedy is to make the cost explicit upfront, so that proposers can make an informed commitment.

Pattern B — re-route to existing entities. A second share of proposals are closed by the chairs with the explanation that the request can be handled with existing core entities — via the "Other" manifestation-subtype, a reserved JSON key, or a core-model change. Documented cases include RO-Crate, Research-product-types and Provenance. At the 13 February 2026 meeting one chair described this as a "never-ending moving target": "some things go into the core, some become a formal extension, some are handled by using existing Research Product subtypes." That is a tiered-extension policy the WG already operates in practice, but informally and only from the chair side — invisible to proposers. Sections 7.6 and 9 recommend formalising it.

Pattern C — deferred to a future core version. Some proposals are recognised as legitimate but classified as core-breaking changes rather than extensions and deferred pending a future core revision. Documented cases include Project/Grant restructuring (21 March 2025: "probably a breaking change in core entities, not an extension") and multi-language support (re-routed into core for "long-term implementation with broader community engagement"). Unlike Pattern A, the proposal is not rejected; unlike Pattern B, it cannot be resolved with existing entities. The proponent is left waiting for a core-version process that has no defined timeline or governance path — or, as in the case of `is_cited_by`, working around it at extension level while the general solution remains unresolved.

5.6 SKG-IF sustainability beyond RDA and OSTRails

SKG-IF's long-term sustainability is currently institutionally anchored in the RDA WG, which provides both normative authority that makes SKG-IF a shared standard rather than one consortium's API, and the decision-making body for extension acceptance and core modifications. When a WG closes, RDA provides fallback routes [§10.4], but these are designed for static recommendation documents, not for a versioned, technically complex framework on which adopters are building. The degree to which SKG-IF can sustain its governance functions without RDA as a platform and projects actively contributing to SKG-IF, such as OSTRails, is not secured and should be cared for.

6. Added-value assessment per component

Section 6 assesses the six artefacts required by §4.1, on value delivered versus effort required. §6.7 covers Widoco HTML documentation and ontology serialisations, both mandated outputs; Widoco is the tool the extension repo template (`ext-tmpl`) recommends. §6.8 covers the Graffoo diagram, not formally required by the extension template, but used by the core and `ext-ra-skg` documentation, making it a community convention in practice. §6.9 covers the `dev-check.sh` validation pipeline, the `ext-srv` authors' own initiative and not part of the official workflow. Ratings are indicative and based on the `ext-srv` experience.

6.1 OWL ontology (`srv.ttl`)

⁵ FAIRsharing (April 2025), KGI4NFDI (October 2025), Facilities and RAiD/Project (both from the OSTRails PSC list, March 2025), geolocation and timestamps (November 2025), and Indicators for OS-aware research assessment (postponed October 2023, never resumed).

Value delivered: semantic reasoning, class/property inheritance, and downstream tool support for consumers that actually use OWL. In the OSTRails community surveyed so far, no implementer consumes the `.ttl` at reasoning level; all consume the JSON-LD context.

Effort required: the highest of any component. OWL authoring carries a triple annotation burden that is not apparent from the extension repo template (ext-tmpl). For every class, the extension author must maintain: (1) standard OWL axioms (domain, range, `subClassOf`); (2) a `dc:description` annotation in a specific structured format ("The properties that can be used with this class are: * prop -[cardinality]-> type") that the SKG-IF SHACL extractor parses to generate SHACL shapes; and (3) an `rdfs:label` annotation in the format "externalName (SKG-IF labels: json_alias)" that maps the OWL class or property to its JSON-LD alias. These are SHACL-shaped and JSON-LD-shaped concerns embedded inside the OWL file, not OWL-native concerns, and the review bottleneck of Section 5.4 lands on all three layers simultaneously. The problem is OWL creation, not OWL consumption; there is no case for reasoning and inference.

Verdict: the ontology is useful as a normative source from which other artefacts can be generated; whether it must be authored by extension proponents is a separate question (see Sections 7.1, 7.3, 7.6).

6.2 SHACL shapes (shacl.ttl)

Value delivered: instance-data validation, the component with the clearest operational pay-off: implementers do want their data validated.

Effort required: in ext-srv the shapes are auto-generated from the ontology by shacl-extractor, which shifts the visible cost onto the ontology. The concrete form that cost takes is maintaining the cardinality constraint annotations inside the OWL file. The SHACL extractor reads `dc:description` annotations on `owl:Class` definitions (format: "The properties that can be used with this class are: * prop -[N]-> type") and re-materialises them as formal SHACL shapes. This is a SHACL-extractor-specific convention, not standard OWL; extension authors must learn and maintain it as a third annotation layer alongside OWL axioms and `rdfs:label` JSON-LD aliases (see §6.1). These constraints are therefore part of the ontology author's responsibility even though they are SHACL-shaped, not OWL-shaped, concerns, and the opacity of this generation step was flagged even in the OSTRails-internal extension analysis. A further open question: if the generated SHACL checks only the cardinalities declared in the OWL, that may be under-constrained for real validation needs: a permissive SHACL file will effectively pass everything. Sharpening the shapes beyond what the extractor emits by default is a skill-level question flagged here for Section 8.1 to pick up.

Verdict: SHACL stays; the real design questions are whether it must be generated from OWL, and whether cardinality-only generated shapes are specific enough for production use.

6.3 JSON-LD context (skg-if.json)

Value delivered: the most consumed artefact in the stack. Every OSTRails implementer that has produced SKG-IF output has done so via the JSON-LD context, layered with `@base` and the extension's own additions. Effort required: moderate; the context has to stay in sync with the ontology and overlay but is itself compact.

Verdict: the highest value-to-effort ratio of the six components – a reading corroborated by the multi-community commentary, which consistently showed a preference for JSON-LD and API-based exchange over ZIP bundles [§10.1].

On the cost side: the context imports terms from upstream ontologies (FaBiO, foaf, and similar) that are often unfamiliar to research-community implementers, and in places the imported label is a poor semantic fit for what the property locally models. This is a cost the extension workflow asks implementers to accept.

6.4 OpenAPI overlay (service-overlay.yaml)

Value delivered: exposes the extension in the REST API; value scales with the number of implementers that expose services via HTTP, and the multi-community preference for API-based exchange (Section 2.1) increases this expected value.

Effort required: heavier in practice than its artefact size suggests: the overlay is YAML with strictly significant indentation and has to be edited with knowledge of the full consolidated OpenAPI specification target; its alignment with the ontology, context and the `Service.md` prose is currently a hand-maintained responsibility, where `Service.md` is effectively the prose-side source for the overlay shape, meaning that changes in `Service.md` propagate into the overlay as part of the sync cost.

Verdict: the overlay component is well-placed; the question is whether single-artefact generation (Section 7.3) could cut the hand-alignment cost between the ontology, SHACL, and JSON-LD context; the overlay itself still requires hand-authoring.

6.5 Prose specification (Service.md)

Value delivered: the human entry point for anyone adopting the extension, and non-negotiable in practice. The prose `.md` file is also the artefact that circulates among community experts during design discussions, precisely because it is the format that is cheapest for domain specialists to read, mark-up and argue with; that conversational function is not easily replaced by a generated-documentation artefact.

Effort required: 19 KB of prose whose alignment with the ontology, context and overlay is entirely manual.

Verdict: required and valuable, and a persistent synchronisation liability, but not a good candidate for mechanical generation from structured sources, because the prose does work (community-facing discussion, argumentation, scoping) that generated documentation does not.

6.6 Example set (examples/)

Value delivered: essential for onboarding and for validating the overlay; 120+ `Service` records in `ext-srv` supply both. Effort required: curation cost is real and ongoing: examples are the first artefact to expose misalignment when context or overlay changes. Two data points sharpen the effort picture: the SKG-IF core repository itself carries only a minimal set of examples, which is itself a tell that example curation is expensive to sustain; and the consolidated OpenAPI specification embeds example records inline, adding a further sync surface that is often overlooked when the headline effort is described. Verdict: keep, but treat as generated test fixtures where feasible, not as hand-written documents.

6.7 Widoco-generated HTML and ontology serialisations (both mandated by template)

Both deliverables mentioned here are mandated by the extension template: four OWL/RDF serialisation formats (`.ttl`, `.nt`, `.xml`, `.json`) for content negotiation behind `w3id.org` URI, and an HTML documentation page. Widoco produces both in a single run and is substantially easier than the alternatives the extension repo template (`ext-tmpl`) mentions.

Value delivered: The HTML page is the most accessible entry point for reviewers not comfortable reading Turtle, and the published record of the ontology on the SKG-IF documentation site.

Effort required: moderate per run. Widoco invocation is a separate manual step, not integrated into `dev-check.sh`, and in practice the HTML documentation was regularly out of sync with the current ontology state.

Verdict: the HTML documentation earns its keep as a human-readable entry point and published documentation artefact. The remaining gap is that HTML generation is not part of the `dev-check.sh` pipeline. The alternative serialisations (RDF/XML, N-Triples, JSON-LD) are cheap by-products of the Widoco run and worth keeping. For a single-source alternative that generates equivalent documentation alongside all other artefacts, see §7.3.

6.8 Graffoo diagram (`srv.graphml`)

Normative status: Graffoo is not mentioned in the extension template (`ext-templ`). Both SKG-IF core and `ext-ra-skg` documentation use it; it is a community convention but it is not prescribed for extension authors.

Value delivered: Graffoo diagrams give reviewers and community commentators a visual representation of the class/property graph that is substantially easier to read than Turtle. For `ext-srv`, `srv.graphml` has been useful during design discussions, and lowers the cost of bringing a non-RDF-fluent commentator into a design conversation. The value is entirely in discussion and documentation support, not in runtime validation or downstream generation.

Effort required: entirely manual. yEd editor is a graphical tool, not a generator from the ontology; the diagram is authored by hand from `srv.ttl`, and every ontology change (class rename, property addition, domain/range adjustment) obliges a manual diagram update. No SKG-IF tool keeps the Graffoo diagram in sync with the ontology; in practice the diagram lags. The update effort per change is small but it adds yet another manual sync channel (§5.3).

Verdict: keep Graffoo diagrams as an optional documentation artefact embedded in the data-model pages, especially for extensions with a complex class hierarchy. For a generator-based alternative such as LinkML, see §7.3.

6.9 Validation pipeline (`dev-check.sh`)

Value delivered: the 8-step validation pipeline (§4.3) provides the cross-artefact consistency checking that the official SKG-IF workflow does not. It validates the 120+ example records are validated against the SHACL shapes, and checks context / overlay / ontology alignment after every change. Without it, the manual synchronisation cost would be substantially higher. It was created by the author, with assistance from a coding AI tool, specifically to manage the multi-artefact dependency problem that emerged during `ext-srv` development.

Effort required: the pipeline itself is a shell script (~500 lines) that wraps existing tools (riot, shacl-extractor, Spectral, Prism, a set of Python alignment checks). Creating it required understanding the dependency order of all artefacts and encoding it in a reproducible sequence. Maintenance cost is real: when a tool version changes, or when a new artefact is added, the pipeline needs updating. The pipeline is not self-documenting; understanding what step 5 checks requires reading the script.

Verdict: the pipeline is the highest-leverage `ext-srv`-authored artefact: it earns its creation cost many times over through reduced manual checking. However, it should be a SKG-IF-framework-provided tool, not an extension-author-authored one. Many of the checks it performs are common to every extension and could be provided as a parameterised reference script (namespace prefix, file paths,

example count as inputs).⁶ Requiring every extension author to write their own consistency pipeline is unreasonable; the framework should supply a reference pipeline that extension authors can run and optionally extend.

7. Alternative / lighter approaches

The alternatives below are not presented as a replacement of the SKG-IF stack but as routes the framework could formally accommodate. Each is assessed on implementation cost, interoperability with the existing SKG-IF stack, tool support, discoverability, and governance compatibility.

7.1 JSON-LD context + SHACL only (no mandated OWL)

If SHACL is authoritative for validation and JSON-LD for serialisation, the OWL layer becomes optional rather than mandatory for an extension. Implementers whose tooling does not consume OWL would be freed from the highest-cost component (Section 6.1). Interoperability with the core model is preserved because extensions would still be expressed as JSON-LD compatible with the core context, together with an extension context file that introduces the new properties, relations and values (these are aliases to already existing internal or external IRIs).

Validation, serialisation and human-readable documentation are fully recoverable without OWL; what is lost is the formal `srv:Service` class declaration with its `rdfs:subClassOf` chain into the core hierarchy, and any subproperty links. No current OSTrails consumer uses either. Where terms from existing vocabularies already cover a needed concept — `foaf:Organization`, `schema:name`, `dcterms:description` — the extension context references those IRIs directly, inheriting their published semantics. The one authoring change is that SHACL shapes must be written directly rather than generated from annotated OWL via `shacl-extractor`. Governance compatibility requires a defined minimal-extension tier (see Section 7.6).

This route is fully self-contained: the extension references core URIs via the existing context-layering mechanism, with no OWL import chain and no coordination required with the core team.

This route is only genuinely lighter where the extension can be expressed through existing vocabulary IRIs; introducing a new namespace commits the author to the same IRI-management obligations as a formal ontology, with or without a `.ttl` file (see §7.6 for the minimal case where no new entity type or IRI is introduced).

⁶ What cannot be fully generalised is extension-specific validation: custom property sets, anything that depends on the extension's own semantic choices. Those would remain the author's addition on top of the reference pipeline

Minimal-extension tier — JSON-LD context + SHACL only

ext-srv without OWL; context + SHACL carry the semantic load (report §7.1, OWL-replacement analysis)

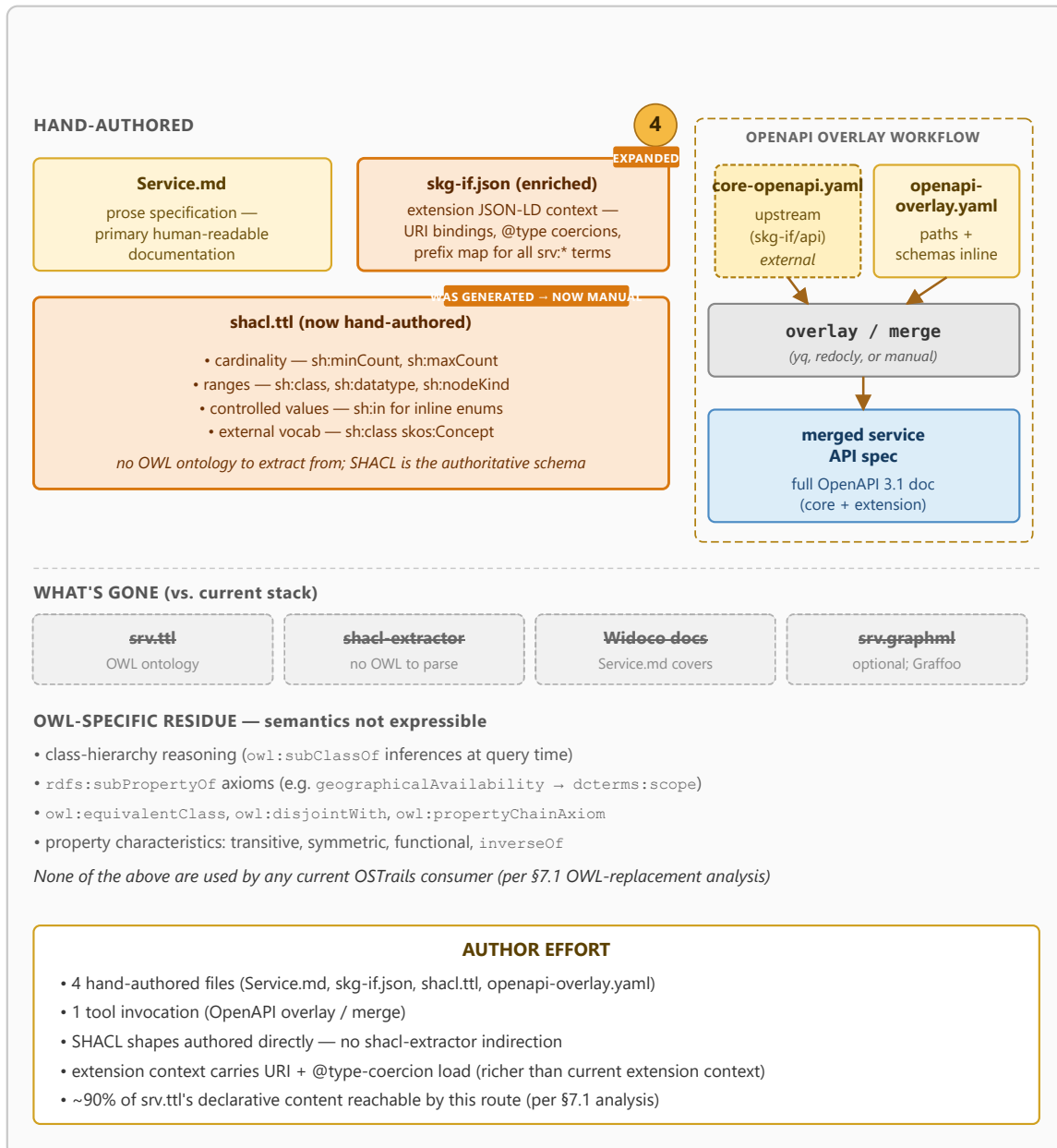


Figure 2 Minimal-tier stack — JSON-LD context and SHACL shapes, no OWL

7.2 Application-profile pattern (DCAT-AP style)

An application profile reuses existing ontologies and publishes constraints and guidance (DCAT-AP style) rather than creating a new ontology. For SKG-IF, a profile approach means extensions constrain and recombine existing entities and properties rather than introducing new ones.

The ext-srv proponents rejected this path: a Research Product describes a versioned deliverable; a Service describes a running infrastructure component with a life-cycle status that changes while the identifier stays fixed. Collapsing the second into the first loses the semantics that motivated the extension. Getting ext-srv through the WG acceptance step required insisting on this conceptual separation; that discussion is itself part of the workflow costs documented in section 5.

On the extension / core-separation question: a profile is by definition about the core. It can be maintained in a separate repository, but its meaningfulness is tied lock-step to the core version it profiles: a core change that affects the profile's target classes requires a coordinated profile update. There is no extension-specific entity namespace. Although the extension is separable in repository terms, it is more tightly coupled in versioning terms than a new entity extension.

7.3 LinkML-driven generation

A single YAML schema in LinkML generates JSON-LD context, SHACL shapes, OWL, JSON Schema and documentation. The synchronisation cost identified in Section 5.3 is largely eliminated because artefacts become generated outputs of a single source. LinkML is for example in active production use in bio-data (NMDC, Biolink Model). For SKG-IF the core migration cost would be serious, but every new extension after migration would avoid the synchronisation cost this report documents. To make the concrete artefact size visible, a first-pass LinkML YAML sketch for the Service entity is given in Appendix A. The generated documentation is bare by default: one page per class and slot — but LinkML supports per-slot description, comments, and examples fields that `gen-doc` picks up and renders.

On the extension / core-separation question: the generated artefacts integrate identically to §7.1: a consumer sees the same JSON-LD context, SHACL and OpenAPI overlay,⁷ regardless of whether they were hand-authored or automatically generated by `gen-owl` / `gen-shacl` / `gen-jsonld-context` / `gen-json-schema`. At authoring time, however, the single-source benefit is strongest when the core also publishes a LinkML schema that the extension can import. Without that, the extension schema has to declare upstream classes as URI stubs, losing internal type-checking at the extension/core boundary, though the generated SHACL shapes still validate data correctly. Appendix A carries the detail for readers who want it.

This makes §7.3 a stronger candidate as a project-wide decision (core + extensions all LinkML) than as an isolated per-extension choice: at the artefact level it separates as cleanly as §7.1, at the authoring level it works best when the surrounding ecosystem has made the same tooling choice.

⁷ The overlay is always hand-authored.

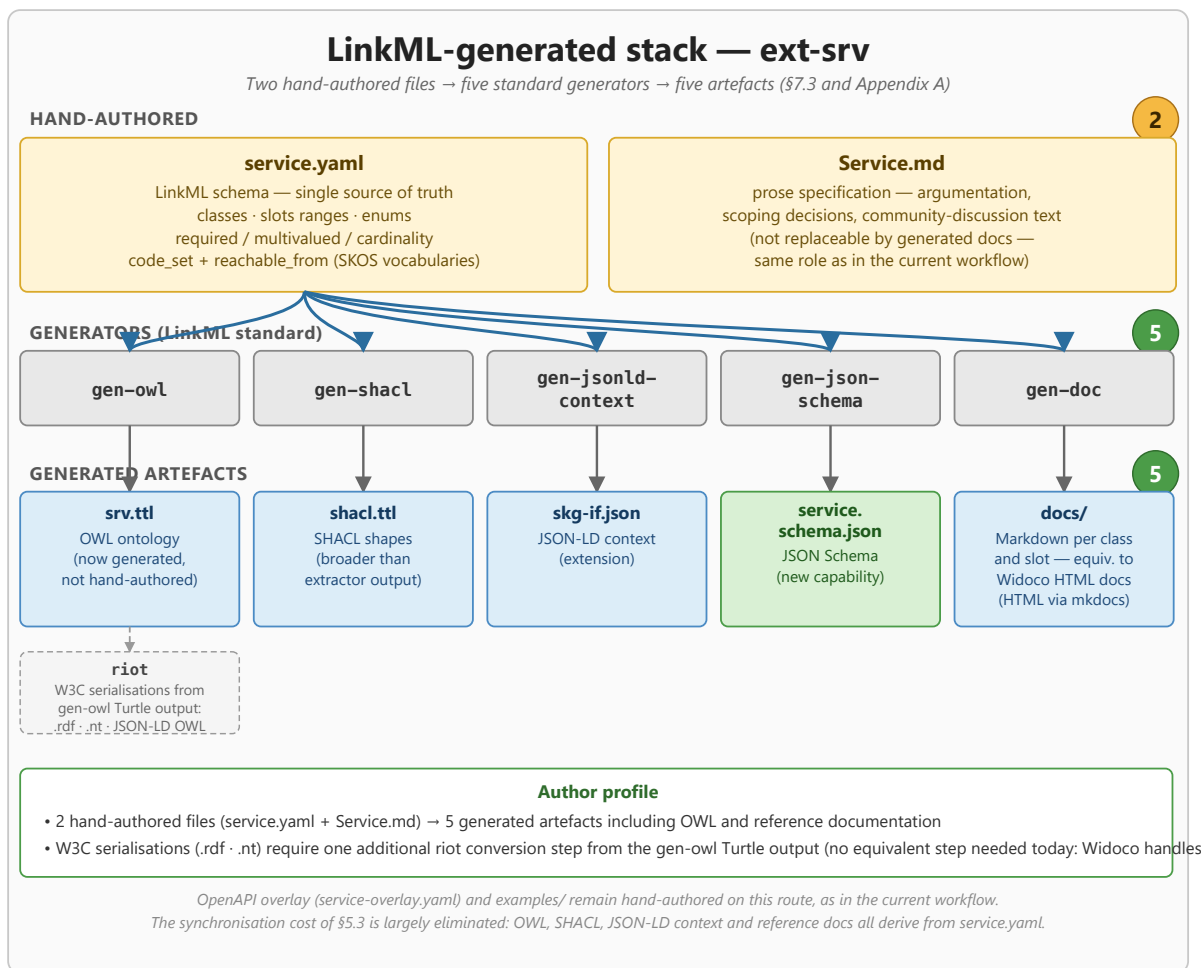


Figure 3 LinkML-generated stack — two hand-authored files, five generated artefacts

7.4 Schema.org-style extensions

A note on terminology before the assessment. There are two quite different things that can be meant by "using schema.org" for an extension and conflating them leads to confusion. Route (1): referencing schema.org IRIs in the JSON-LD context or SHACL shapes of a SKG-IF extension. This is already done in practice: ext-srv reuses `schema:keyword`, `schema:isAccessibleForFree` and `schema:url` in its current context, and is fully compatible with the SKG-IF workflow without any additional governance step. Renaud Duyme's (ESRF) vocabulary analysis for instruments, which maps PIDINST and CDIF concepts to `schema:IndividualProduct` and related terms, follows the same route and is intended as input to a formal SKG-IF extension proposal. Route (2): authoring an extension under schema.org's own extension procedure (pending schema.org or a community namespace under schema.org governance). This is a different interoperability framework entirely; the extension sits outside SKG-IF governance and would require a crosswalk to participate in SKG-IF queries.

For route (1) the extension/core-separation question is straightforward. Schema.org IRIs are stable, externally published references; the extension context and SHACL shapes reference them by IRI, and no coordination with the schema.org organisation is needed beyond the conventions that already

govern third-party vocabulary reuse in SKG-IF. The one design question is which schema.org terms to alias and whether the existing core-context entries already cover the need.

7.5 OpenAIRE Graph route (crosswalk to OAG, not extension of SKG-IF)

A route sometimes confused with SKG-IF implementation: a community participates in the OpenAIRE Graph (OAG) by exposing metadata via OAI-PMH or comparable protocols for OpenAIRE to harvest; the aggregated metadata then becomes accessible through OAG's SKG-IF API and `explore.openaire.eu`. This is neither an extension nor an implementation of SKG-IF: it does not require the artefacts in §4.1 and does not go through WG governance. Several OSTRails crosswalk activities target OAG rather than SKG-IF directly. A community that has completed an OAG crosswalk gains visibility in SKG-IF via OpenAIRE but has not produced a SKG-IF extension or its own SKG-IF-compliant endpoint.

Community-specific metadata that does not match SKG-IF core and available extensions will not surface via OAG's SKG-IF API; exposing it requires extensions added and implemented by OpenAIRE. However if there is no need of special metadata representations beyond what is covered by the SKG-IF core, and there is already an `OAI-PMH` endpoint available, this is a relatively cheap way for publishing your metadata in a SKG-IF interoperable way.

7.6 A lightweight track for minimal core modifications and extensions

SKG-IF currently has one extension workflow for all cases, while there are no formal rules for modifying the core yet. A formally recognised lighter tier would cover two kinds of addition that do not warrant the full stack. First, extensions to existing enumeration values for instance to add additional types of Research Product or support additional types of relations such as adding a `is_cited_by` relation type to the already existing `cites` relation. Such modifications are backwards-compatible by definition and need only a context entry and documentation update. Secondly, additions of new properties on existing entity classes that do not require minting a new IRI need a context entry and additional SHACL shapes, but no new OWL class, no Widoco documentation, and no Graffoo diagram. `ext-ra-skg` is the clearest illustration of the gap: it adds no new entity, only attributes on existing classes, yet went through the full OWL and SHACL authoring workflow. The lighter tier would have been the appropriate route.

The caution is the same as in §7.1: this tier is only valid where the semantic fit with existing entities is genuine. It is not a way to avoid the full workflow when a new entity is actually warranted; that boundary needs explicit discussion or chair-level decision per case.

8. Current discussion points

The discussion points listed in this section are the opinion of the author but should also be a fair representation of different points of view that are voiced in the WG discussions, including that of the WG chairs who bear responsibility for the framework's coherence and its pre-OSTRails commitments. The author focuses on the needs for using the SKG-IF by the research communities, going beyond its initial bibliographic scope.

The SKG-IF JSON-LD context and the API layer deliver real interoperability value: every OSTRails implementer that has produced SKG-IF output uses the context, and the API specification gives communities a common query interface that did not exist before. A shared six-entity vocabulary, however incomplete and focussed on bibliography, creates common ground across CESSDA, CLARIN, OpenAIRE, TeslaRIS and OpenCitations that would otherwise require bilateral mapping agreements. The SHACL layer has not yet been tested end-to-end in production, but the potential benefits are

clear: machine-actionable validation is what research-community implementers actually want from a framework, and SKG-IF is positioned to deliver it.

The current workflow for creating extensions is however too costly for the benefits it currently delivers. The OWL ontology, the major cost factor, is used in practice not for reasoning or inference, but mainly to specify cardinality constraints for the shacl-extractor. The core ontology itself is incomplete in this regard, and no OSTRails implementer uses it at reasoning level.

As for using the ontology to ground the SKG-IF concepts, there is a heavy preference to use bibliographically oriented ontologies (FABIO/FRAPO/SPAR), with limited coverage of schema.org, DCAT-AP and the EOSC profiles. There is encouragement for the extension authors to reuse the vocabularies used in the core, so this preference will probably remain. The EOSC profiles embody substantial community expertise in data and compute management; drawing on them would benefit interoperability with larger infrastructure initiatives.

Adopting a lighter-tier extension is not free: where lighter-tier extensions compromise extension-domain semantics, they can hamper downstream adoption and correct application of SKG-IF. The open question is whether the strictness of the SKG-IF core ontology is itself justified, given that several of its own entities (Venue, Grant, Data Source) are structurally open to the “should-have-been-a-relationship” critique discussed earlier and below. A consistent position would either hold the core strict and accept that extensions must match its strictness or loosen both. Our recommendation in Section 9 sits in the middle: precise semantic modelling and broad interoperability pull in opposite directions, and the choice of how strict to be should be made consciously rather than by default.

Proponents of the current workflow have a principled position with respect to the necessity of the OWL ontology layer. Linking new entities and properties to the wider ontology world, preferably using ontologies the core already draws on, requires semantic expertise that currently sits with the chairs; community participation (CLARIN, CESSDA, etc.) can share the review load, but cannot be expected to reach beyond assessing domain applicability. The RDA SKG-IF WG output (Mannocci, Peroni & SKG-IF WG, 2025) is the formal WG deliverable that describes the framework components and the extension process but offers no argument for choices such as the necessity of an ontology layer.

In the cross-community commentary of SKG-IF's fit, Keith Jeffery (who was initially heading community involvement in OSTRails and later continued as an advisor) raised structural objections: Grant and Venue are arguably better modelled as typed relationships than as first-class entities, making the core not yet a true graph model in the strict sense. On identifiers, UUIDs were suggested as a cleaner alternative; CURIEs were adopted for `local_identifier`, but with some debate still ongoing. Similar challenges can be expected as the framework is applied more broadly. The discussion has since subsided; a structural overhaul is beyond current resource capacity but the critiques stand as a useful reminder that SKG-IF is a workable starting point, not an optimised graph model.

A recurring friction is the resistance to broadening SKG-IF beyond a publication-centric perspective. For instance, the SKG-IF accommodates relations from the publication outward, a paper cites a dataset or service but resists the inverse perspective where a dataset or service is the centric entity and expresses which publications cite it (`is_cited_by`). In a research-infrastructure or dataset-centric context, the latter is the natural and operationally useful direction: the dataset provider knows and cares which papers cite their dataset; the paper producer does not systematically enumerate the services they used. The discussion is ongoing and may yet be resolved.

The WG chairs' position is basically sound: all additions must remain consistent with earlier agreements because adopters are presumed to be relying on backwards compatibility. This is a legitimate stance if the adopter base is substantial, but only a few KG initiatives have anything to show: OpenCitations has a few example records and a limited staging endpoint; OpenAIRE and TeslaRIS[§10.7] are the active production adopters. OpenAIRE, leading OSTRails, was well-placed to push for improvements rather than adopt a conservative point of view for so few committed adopters. Conformance with the core specification is uneven: for example, as of May 2026, neither implementation fully conforms to the `local_identifier` specification. The persistence of `local_identifiers` across queries remains an open practical concern. As of June 2026, DataCite, Crossref, and OpenAlex — among the communities whose practices most shaped the model — have not yet produced SKG-IF endpoints. Others have: OpenAIRE, TeslaRIS, OpenCitations, and research infrastructures CESSDA and CLARIN/VLO, at varying levels of completeness and maturity. The picture continues to develop as OSTRails concludes.

9. Recommendations

The recommendations below aim to reduce costs and increase transparency of the extension-creation workflow. They address both the technical layer and governance layer. They are ordered from lowest to highest institutional friction. Items 1–4 are largely technical; items 5–11 touch governance and are the ones that most directly address Section 5.4.

1. Define a “minimal extension” tier (Section 7.6) in the SKG-IF interoperability framework documentation: JSON-LD context plus SHACL shapes plus examples, without mandating a new OWL ontology. For example, `ext-ra-skg` adds no new class; it uses a separate SHACL file targeting an existing entity with no Widoco documentation or Graffoo diagram required.
2. At a still lighter level, two OSTRails cases require neither a new entity nor a new property URI. For RO-Crate (OSTrails/Poznań), the existing research-product entity suffices with a new product-type value and `dcterms:isPartOf` admitted to the context. For `is_cited_by` (OSTrails/CESSDA), `cito:isCitedBy` already exists in CiTO and needs only a context entry and a SHACL constraint. Both concepts are defined in accepted external vocabularies; admitting them is backwards-compatible by definition. Such additions should be administrative formalities, not full extension proposals.
3. Adopt a generator-based authoring approach (LinkML or equivalent) for extensions that do require the full stack including ontology extension. The one-time cost is learning the toolchain; the saving is that all downstream artefacts regenerate from a single schema change.
4. Add more CI workflow to the extension template that regenerates derived artefacts (SHACL, Widoco HTML, serialisation formats) on push to the ontology file. The notify-parent infrastructure is already in place across the organisation; extending it to cover artefact regeneration removes the author-discipline requirement for keeping derived files in sync.
5. Acknowledge community-local-model crosswalks to SKG-IF as a parallel activity, not a form of extension. Where such crosswalks exist, link them from the extensions index without folding them into the extension-review workflow.
6. Shift as much review workload as possible into automated checks — SHACL generation, context-compatibility, OpenAPI linting, examples coverage — so that the ontology-literate reviewers the WG can realistically field are reserved for what automation cannot decide. A `dev-check.sh`-style script is the right model; notably, the current `dev-check.sh` was authored by the `ext-srv` proponent, not provided by the SKG-IF framework.

7. Make contributions from both reviewers and proposers explicit. Review and response steps should be listed with target dates, agreed between the WG and the proponent at the start of each proposal.
8. Introduce a pre-submission consultation step so that ontology design issues are surfaced before weeks of work. The consultation can be light-touch, a 30-minute chair conversation, and its existence and outcome should be documented openly.
9. When writing future project proposals, budget for evaluation and extension review time. In RDA WGs, participation is partly voluntary; the constraint is not a project-rules issue since OSTRails participants can charge activity to the project, but a matter of acknowledgement.
10. Document the tiered-extension policy the WG already operates in practice and publish it as part of the extensions index. The policy should guide proposers in selecting the appropriate tier:
 - a. vocabulary additions and backwards-compatible property admissions are administrative formalities;
 - b. core-entity adaptations without new classes qualify for the minimal-extension tier;
 - c. proposals that introduce genuinely new entity types with distinct semantics require the full extension workflow.

Re-routing a proposal to a lighter tier is only valid when the semantic and functional fit is adequate; Service and Instrument are examples where it is not.
11. The WG should publish explicit acceptance criteria for both the extension pathway and the direct core modification pathway, including criteria that address broadening the scope beyond the current bibliographic focus.

10. References

Grouped by source type. Starred items were mined directly in preparing this draft; unstarred items are sources still to be confirmed or consulted in a later pass.

10.1 Source material used in this draft

- * ../ext-srv/creating ext-srv.pptx — author's own summary; basis for Section 5.3 synchronisation-cost framing.
- * ../ext-srv/Service.md, structure.md, README.md, dev-check.sh — ext-srv repository; basis for Sections 4 and 5.2.
- * sources/Comments on the RDA SKG IF model.txt — multi-community commentary (CESSDA, CLARIN, SSHOMP, ESCAPE, PaNOSC-ESRF, Keith Jeffery, Daan). Basis for Sections 2.1, 3, 5.1, 8.
- * sources/SKG-IF Service extension.txt — ext-srv proposition with the six-vocabulary property inventory; basis for Section 5.1 and 7.4.
- * sources/Pilots-implementation-actions — extract.md — OSTRails Use Case III; basis for Sections 2.2 and 7.5.
- * sources/SKG-IF Rolling agenda and minutes — Google Docs export — RDA SKG-IF WG rolling agenda/minutes (July 2022 – April 2026, 1130 paragraphs); basis for Sections 2.1, 4.4, 5.4, 5.4.1, 7.6 and 9 item 8. See the derived digest sources/Rolling-agenda-minutes — findings.md.

10.2 SKG-IF official documentation (online)

- **Home** — <https://skg-if.github.io/>
- **Data model** — <https://skg-if.github.io/data-model/>
- **Interoperability framework** — <https://skg-if.github.io/interoperability-framework/>

- **API** — <https://skg-if.github.io/api/>
- **Extensions index** — <https://skg-if.github.io/extensions/>

10.3 GitHub repos (skg-if org)

- **skg-if (org)** — <https://github.com/skg-if>
- **skg-if/data-model** — core OWL ontology (FaBiO-based)
- **skg-if/interoperability-framework** — specification prose
- **skg-if/api** — OpenAPI specification + reference implementations
- **skg-if/context** — core JSON-LD context
- **skg-if/ext-srv** — Service extension (this case study)
- **skg-if/ext-ra-skg** — first existing extension — core-entity adaptation (contrast case)
- **skg-if/shacl-extractor** — generates SHACL from annotated ontology
- **skg-if/extensions** — extensions tracker (issues = proposal pipeline)
- **skg-if/examples** — example JSON-LD data across implementers

10.4 OSTRails and RDA material

- **OSTRails SKG-IF page** — <https://ostrails.eu/skg-if>
- **OSTRails SKG-IF Implementations Registry (Google Sheet, read-only)** — registry of all OSTRails implementations
- **RDA SKG-IF WG Case Statement (2024)** — RDA Case Statement PDF
- **RDA SKG-IF WG output 1-1 (Dec 2025)** — formal WG deliverable
- **RDA SKG-IF WG home** — <https://www.rd-alliance.org/groups/scientific-knowledge-graphs-interoperability-framework-skg-if-wg/>
- Mannocci, A., Peroni, S., & SKG-IF WG. (2025, December 12). Scientific Knowledge Graphs Interoperability Framework (SKG-IF): Enabling Data Exchange for SKGs. Research Data Alliance. <https://www.rd-alliance.org/wp-content/uploads/2025/12/RDA-SKG-IF-WG-output-1-1.pdf>
- **RDA Outputs Processes** (WG lifecycle, maintenance and retirement plan requirement) — <https://www.rd-alliance.org/rda-outputs-processes/>
- **RDA TIGER project** (support for ending/finished WGs; Maintenance Facility at DANS) — <https://www.rd-alliance.org/working-groups/rda-tiger/>

10.5 Lighter-stack precedents

- **DCAT-AP 3.0** — application-profile methodology publishing HTML/RDF/UML/SHACL + JSON-LD
- **LinkML** — one schema → OWL, SHACL, JSON-LD context, JSON Schema, docs. An Open Data Modeling Framework (<https://arxiv.org/pdf/2511.16935>), <https://linkml.io>
- **SHACL and OWL compared** — background for Sections 6.1 / 7.1
- **SEMIC: Application profiles** — application-profile modelling framing (DCAT-AP case)

10.6 Pipeline tools (ext-srv dev-check.sh)

- **riot** — Apache Jena RDF parser/validator: <https://jena.apache.org/documentation/io/>
- **Spectral** — OpenAPI linter (Stopligh): <https://stopligh.io/open-source/spectral>
- **Prism** — OpenAPI mock server (Stopligh): <https://stopligh.io/open-source/prism>
- **Widoco** — OWL documentation generator: <https://github.com/dgarijo/Widoco>

10.7 SKG-IF adopters

- **OpenCitations** — example records in skg-if/examples; staging endpoint at <https://api-stg.opencitations.net/skg-if/v1> (products only)

- OpenAIRE — production API at api.openaire.eu (SKG-IF endpoint); codebase: github.com/openaire
- TeslaRIS — CRIS system (University of Novi Sad); production API at cris.uns.ac.rs/api/skg-if; repo: [sci2zero/TeslaRIS-backend](https://github.com/sci2zero/TeslaRIS-backend)
- CESSDA — research data catalogue; staging API at skg-if-staging.cessda.eu/products; repo: [cessda/cessda.cdc.skg-if.api](https://github.com/cessda/cessda.cdc.skg-if.api), github.com/cessda/cessda.cdc.skg-if.api
- CLARIN — research data catalogue; staging API at clarin-skg.dev.diginfra.org/api/v1/ currently behind VPN; repo: [http://github.com/knaw-huc/ost-clarin-skg](https://github.com/knaw-huc/ost-clarin-skg)

Appendix A — LinkML YAML schema for the Service entity

This is a first draft LinkML schema for the core of the ext-srv Service entity, included to make the artefact size visible and to illustrate the single-source-of-truth principle described in Section 7.3. It is not a production schema and is not a drop-in replacement for `srv.ttl`.

LinkML can generate, from a single YAML schema like the one below, a JSON-LD context (`gen-jsonld-context`), SHACL shapes (`gen-shacl`), an OWL ontology (`gen-owl`), a JSON Schema (`gen-json-schema`), and Markdown documentation (`gen-doc`). Of the six ext-srv artefacts, this covers OWL, SHACL, and JSON-LD context exactly; documentation is generated as structured Markdown rather than a narrative prose specification; and the OpenAPI overlay has no standard LinkML generator and still requires hand-authoring. The synchronisation saving is nonetheless substantial: the three artefacts that currently drive most of the coordinated editing — OWL, SHACL, and JSON-LD context — derive from a single source.

Artefact-size observation for the ext-srv Service entity specifically: the YAML below is ~90 lines for six properties and the service-vs-software distinction. Scaling to the full ext-srv property set (audience, invocation type, life-cycle status, hosting organisation, deployment-of software, keywords, plus the organisation/venue additions) would bring it to an estimated 250–300 lines, against a current `srv.ttl` file of comparable size plus a SHACL file and a JSON-LD context of partially overlapping content. The migration cost is therefore real but manageable; this appendix gives the artefact on which the single-source claim of Section 7.3 can be evaluated.

```
id: https://w3id.org/skg-if/extension/srv/linkml
name: skg-if-ext-srv
title: SKG-IF Service extension – LinkML sketch
description: >-
  First-pass LinkML schema for the Service entity introduced by the
  SKG-IF ext-srv extension. Illustrative only – not a production schema.
license: https://creativecommons.org/publicdomain/zero/1.0/

default_prefix: srv
prefixes:
  linkml: https://w3id.org/linkml/
  srv: https://w3id.org/skg-if/extension/srv/
  skg: https://w3id.org/skg-if/ontology/
  schema: http://schema.org/
  foaf: http://xmlns.com/foaf/0.1/

imports:
  - linkml:types
```

```

classes:
  Service:
    description: >-
      A software or operational service exposed by a research infrastructure
      or community (portal, processing API, catalogue endpoint, ...).
    class_uri: srv:Service
    is_a: skg:ResearchEntity
    slots:
      - local_identifier
      - name
      - description
      - audience
      - invocation_type
      - life_cycle_status
      - hosted_by
      - deployment_of
      - keyword

slots:
  local_identifier:
    identifier: true
    range: uriorcurie
    required: true
    description: Local (w3id- or community-PID-based) identifier for the service.

  name:
    slot_uri: schema:name
    range: string
    required: true
    multivalued: false

  description:
    slot_uri: schema:description
    range: string
    multivalued: false

  audience:
    slot_uri: srv:audience
    range: AudienceEnum
    multivalued: true

  invocation_type:
    slot_uri: srv:invocationType
    range: InvocationTypeEnum
    required: true

  life_cycle_status:
    slot_uri: srv:lifeCycleStatus
    range: LifeCycleStatusEnum
    required: true

  hosted_by:
    slot_uri: srv:hostedBy
    range: Organisation

```

```

    multivalued: false

  deployment_of:
    slot_uri: srv:deploymentOf
    range: uriorcurie
    description: URI of the software whose running instance this service is.

  keyword:
    slot_uri: schema:keywords
    range: string
    multivalued: true

enums:
  AudienceEnum:
    permissible_values:
      researchers: {}
      infrastructure_operators: {}
      funders: {}
      general_public: {}

  InvocationTypeEnum:
    permissible_values:
      http_api: {}
      web_ui: {}
      cli: {}
      library: {}

  LifecycleStatusEnum:
    permissible_values:
      planned: {}
      beta: {}
      production: {}
      deprecated: {}
      retired: {}

```

Running the LinkML generators against the sketch above produces: a JSON-LD context with `srv:*` term mappings; a SHACL shape with required/multivalued cardinalities for the slots; an OWL ontology with class and property declarations; a JSON Schema; and a Markdown documentation tree.